Experiment 3

Title

Create a dynamic To-Do List where users can add, mark as complete, and remove tasks using JavaScript & DOM.

Theory

- A To-Do List application is a basic yet powerful example of a dynamic web application. It allows users to manage daily tasks by adding new tasks, marking tasks as completed, and removing tasks when they are no longer required. Such applications demonstrate the use of client-side scripting and real-time interaction without reloading the web page.
- The To-Do List application uses HTML for structure, CSS for presentation, and JavaScript for dynamic behavior. JavaScript interacts with the web page through the Document Object Model (DOM), making the page responsive to user actions.

A. **HTML (HyperText Markup Language)**

HTML is used to define the basic structure of the To-Do List application. It provides elements such as input fields, buttons, lists, and containers that form the user interface. In a To-Do List:

- Input elements are used to accept task details from the user
- Buttons are used to trigger actions like add or delete
- List elements are used to display tasks

Semantic HTML elements improve readability, accessibility, and maintainability of the application.

B. **CSS (Cascading Style Sheets)**

CSS is used to enhance the visual appearance of the To-Do List. It controls layout, colors, fonts, spacing, and alignment. CSS is also used to visually distinguish completed tasks, for example by applying a strike-through effect or changing text color. Proper styling improves user experience and clarity.

C. **JavaScript**

JavaScript is a client-side scripting language used to make web pages interactive. In a dynamic To-Do List application, JavaScript performs the following roles:

Captures user input

Handles user events such as button clicks

Updates the task list dynamically

Applies logic for task completion and deletion

JavaScript enables changes to occur instantly without refreshing the page.

D. **Document Object Model (DOM)**

The Document Object Model (DOM) is a programming interface that represents an HTML document as a tree-like structure of objects. Each HTML element becomes a node in the DOM. JavaScript can access and manipulate these nodes to dynamically change the content and structure of a web page.

In a To-Do List application, DOM manipulation is used to:

- Create new task elements dynamically
- Insert tasks into the list
- Modify task appearance when marked as complete
- Remove tasks from the list

E. **Event Handling**

Events are actions performed by the user such as clicking a button, typing text, or selecting an option. JavaScript uses event listeners to detect and respond to these actions.

In the To-Do List application:

Clicking the "Add" button triggers task creation

Clicking a checkbox or task marks it as completed

Clicking a delete button removes the task

Event handling is essential for creating interactive applications.

F. **Dynamic Task Management**

The dynamic nature of the To-Do List means that tasks are managed at runtime. Tasks are added, updated, or removed directly in the DOM.

This allows:

- Immediate visual feedback
- Efficient task handling
- Improved performance compared to page reloads
- Each task is treated as an independent DOM element, making management simple and scalable.

G. **Task Completion Feature**

Marking a task as complete is an important feature of a To-Do List. This is achieved by updating the task's state using JavaScript and modifying its visual style using CSS. Completed tasks help users track progress and organize their work efficiently.

H. Task Removal Feature

The task removal feature allows users to delete unwanted or completed tasks. JavaScript removes the corresponding DOM element from the task list. This operation demonstrates effective DOM manipulation and memory management on the client side.

Procedure :

1. Create HTML structure:

```
<input id="task" type="text">
<button onclick="addTask()">Add</button>
<ul id="list"></ul>
```

2. Write JavaScript:

```
<script>
function addTask() {
  let task = document.getElementById('task').value;
  let li = document.createElement('li');
  li.innerText = task;
  li.onclick = () => li.remove();
  document.getElementById('list').appendChild(li);
}
</script>
```

3. Save and run in browser.

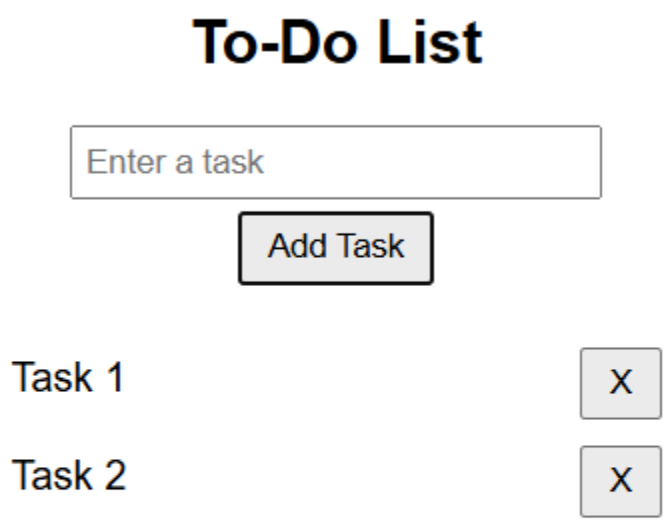
Code:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>To-Do List</title>

  <style>
    body {
      font-family: Arial, sans-serif;
      text-align: center;
      margin-top: 50px;
    }
```

```
    input {
      padding: 5px;
      width: 200px;
    }

    button {
      padding: 5px 10px;
      margin-top: 5px;
      cursor: pointer;
    }

    ul {
      list-style: none;
      padding: 0;
      margin-top: 20px;
    }

    li {
      display: flex;
      justify-content: space-between;
      align-items: center;
      width: 260px;
      margin: 5px auto;
    }

    .completed {
      text-decoration: line-through;
      color: gray;
    }
  </style>
</head>

<body>

  <h2>To-Do List</h2>

  <input type="text" id="taskInput" placeholder="Enter a task">
  <br>
  <button onclick="addTask()">Add Task</button>
```

```
  <ul id="taskList"></ul>

  <script>
    function addTask() {
      const input = document.getElementById("taskInput");
      const task = input.value.trim();

      if (task === "") return;

      const li = document.createElement("li");

      const span = document.createElement("span");
      span.innerText = task;
      span.onclick = () => span.classList.toggle("completed");

      const delBtn = document.createElement("button");
      delBtn.innerText = "X";
      delBtn.onclick = () => li.remove();

      li.appendChild(span);
      li.appendChild(delBtn);

      document.getElementById("taskList").appendChild(li);
      input.value = "";
    }
  </script>

</body>
</html>
```

Output:



Conclusion:

The dynamic To-Do List was successfully implemented using JavaScript and DOM manipulation. Users can **add new tasks**, **mark tasks as complete**, and **delete tasks** in real-time without page reload. The project demonstrates the use of event handling, array management, and dynamic HTML updates, providing practical understanding of interactive web applications**.**

Question:

1.What is an event listener? Give an example.

An event listener is a function that waits for an event (like a click or key press) and runs code when the event happens.

Example:

```
button.addEventListener("click", function() {
  alert("Button clicked!");
});
```

2. What is the difference between innerHTML and textContent?

`innerHTML` gets or sets HTML content (including tags).

`textContent` gets or sets only the text (ignores HTML tags).

3.What is the difference between synchronous and asynchronous JavaScript in this context?

Synchronous code runs one line at a time and waits for each task to finish before moving on.

Asynchronous code allows tasks (like fetching data) to run in the background without stopping the rest of the code.